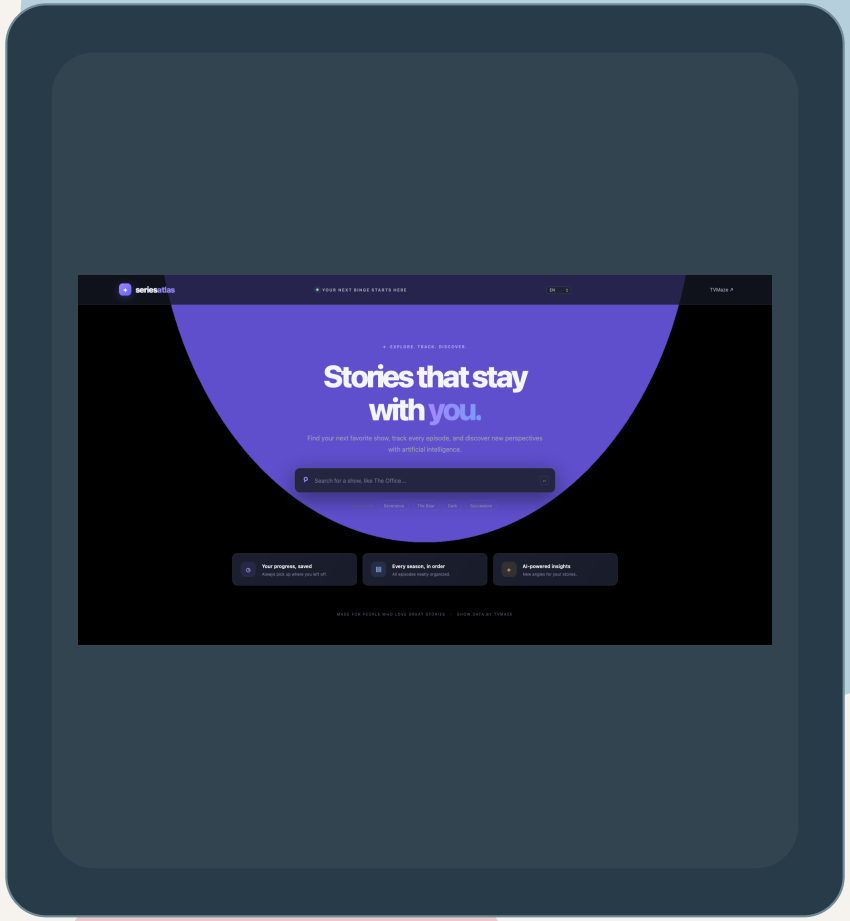
BEECROWD · TECHNICAL CHALLENGE

Series Atlas

Discover series. Follow stories.
Find the context.

A full stack experience with React, FastAPI, PostgreSQL and AI.

Rafael Henter · Full Stack Developer



A product for stories that stay with you.

A continuous journey for every series fan

Discovery, memory and context in one clean flow.

01 · DISCOVERY

Find the next story

TVMaze search with posters, genres and release year.

02 · TRACKING

Keep your place

Seasons, episodes, progress and comments in one view.

03 · CONTEXT

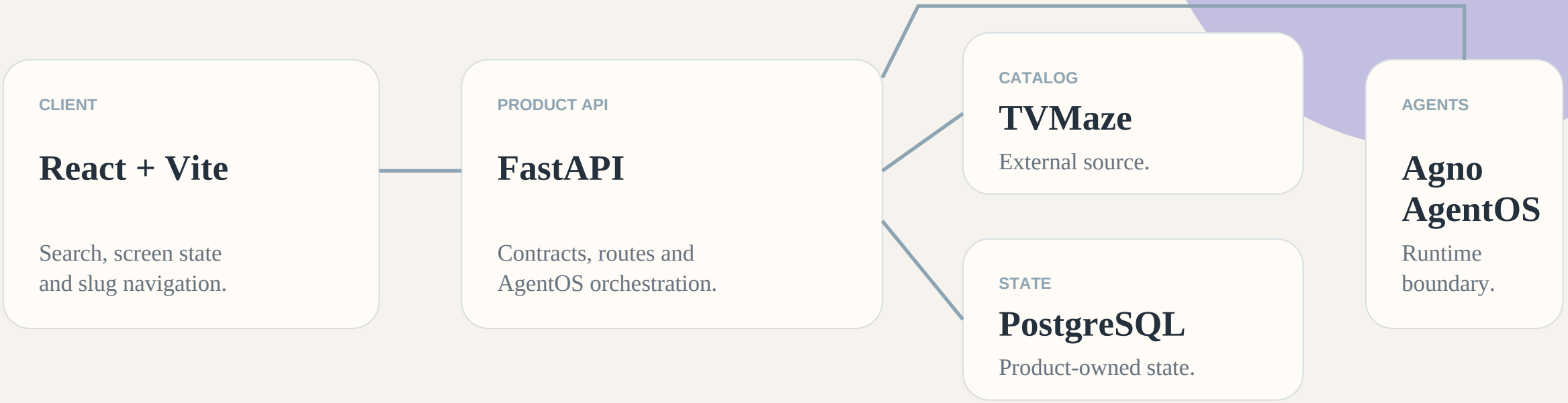
Understand more

A concise insight grounded in the story and community.

Every action leads to the next natural step.

A modular monolith architecture

Few moving parts, clear responsibilities and a simple surface to evolve.



A separate frontend keeps visual iteration independent.
FastAPI gives the product one API boundary for agents, integrations and persistence.

Why the frontend is a separate application

The UI can evolve independently while the API remains the stable product boundary.

SEPARATION

Independent UI work

Fast visual iteration without mixing presentation and domain rules.

REACT + VITE

Fast feedback loop

Components keep behavior composable and testable.
Vite keeps builds lightweight.

RESPONSIBILITIES

Focused modules

Pages, components, API services, state and i18n have clear homes.

Clear boundaries reduce the cost of change.

Why FastAPI is the right boundary for AgentOS

The backend turns agent capabilities into explicit, typed product behavior.

FASTAPI

One product API

Pydantic contracts, validation, explicit errors and OpenAPI.

AGNO + AGENTOS

Agent capabilities

Reusable workflows, sessions, tools and an observable runtime.

MODEL PROVIDERS

Replaceable runtime

Ollama for local control or Hugging Face for hosted models.

The UI consumes a capability; it does not own the agent runtime.

Why PostgreSQL and SQLAlchemy

The application owns user state, so persistence needs to be durable, relational and explicit.

POSTGRESQL

Durable product state

Comments, progress and browser identity need consistency beyond a page refresh. Relations, constraints and transactions protect the experience.

SQLALCHEMY

Explicit data access

Typed models and composable query objects keep data access readable instead of giant SQL strings. Python expressions are easier to test and evolve.

External catalog data stays external; PostgreSQL stores only what the product creates.

Quality is shared by backend and frontend

Tests and internationalization protect the product as responsibilities grow.

TESTS

Behavior over implementation

Backend tests cover API contracts, insight validation and service boundaries.
Frontend tests cover rendered pages, user flows and language selection.

INTERNATIONALIZATION

One UI, two languages

English and Brazilian Portuguese share the same components through LanguageProvider.
The API validates en and pt-BR for AI prompts.

Separation makes testing and localization incremental instead of cross-cutting rewrites.

Each source keeps what it does best

External data stays external; product state stays owned and explicit.

TVMAZE

Public catalog

Shows, seasons, episodes, summaries and images. Adapter normalizes data.

AGENTOS

Agent runtime

Agno + AgentOS expose workflows, sessions and tools. Providers stay replaceable.

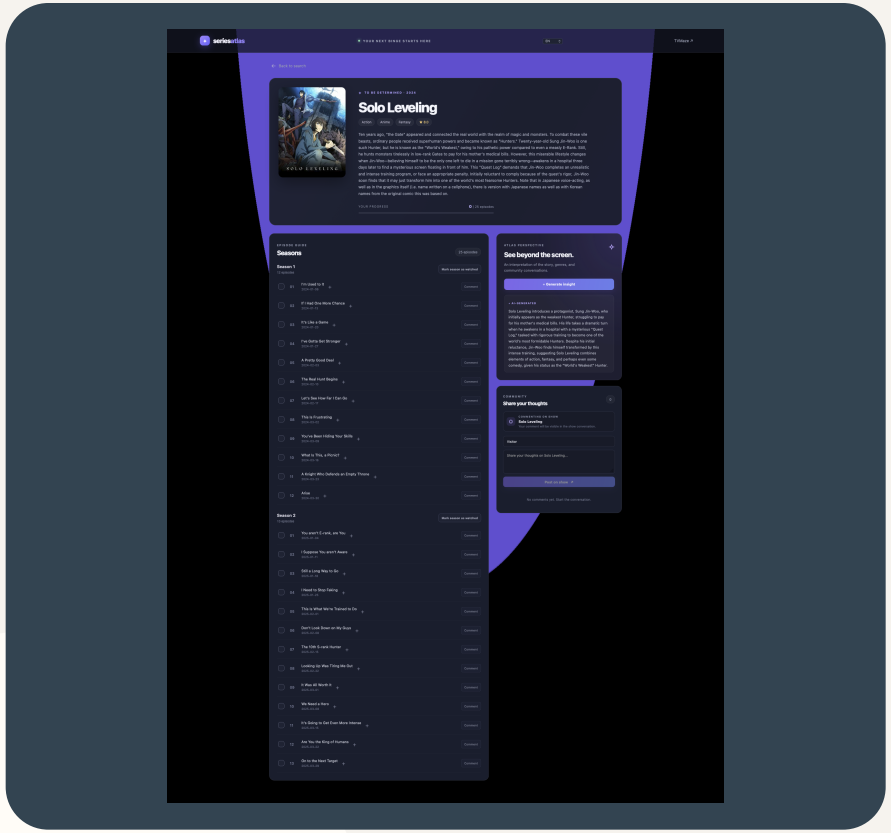
POSTGRESQL

User state

Comments, progress and anonymous browser identity. SQLAlchemy keeps access explicit.

The complete experience fits on one screen

The detail page brings together the decision to watch, track, discuss and understand.



01 **Context**

poster, summary and progress

02 **Pacing**

organized seasons and episodes

03 **Community**

comments linked to the show or episode

04 **Perspective**

insight generated from context

AI with limited context and controlled output

Agno + AgentOS organize the agent layer while Ollama or Hugging Face provides the model runtime.

INPUT

Summary
Genres
Comments

PROMPT

Clear scope
Interface language
No invention

PROVIDER

Ollama or Hugging Face

OUTPUT

Short insight or fallback

Explicit prompts, replaceable providers and safe failure behavior.

State and navigation survive the next visit

The experience continues where the user left off, without an account or a new search.

BROWSER

HttpOnly cookie

Anonymous identity for comments and progress.

PUBLIC ROUTE

/solo-leveling

Slug resolves the show and supports direct reload.

DATABASE

Progress + comments

State is restored without repeating the search journey.

A small UX decision makes the product shareable.

Ready to be evaluated as product and code

The delivery is organized for a fast demo and an objective technical review.

Frontend	React + Vite, responsibilities and i18n
Backend	FastAPI, Pydantic and AgentOS integration
Data	PostgreSQL + SQLAlchemy for durable state
Quality	Frontend and API tests with isolated providers

Health: /health · Docs: /docs · AgentOS: /os

A small foundation with room to grow

Clear boundaries make the next level visible without hiding the demo's limits.

REVIEW MAP

Code with clear homes

routes.py · orchestration and contracts
tvmaze.py · integration and normalization
insights.py · provider and fallback
tests/ · essential behaviors

NEXT STEPS

From demo to production

Authenticated identity
Operations and observability
Measured scale when latency justifies it

Discovery with memory. Context when it matters.

Questions?

Let's talk about the product, the trade-offs and the next episode.

PRODUCT

Discovery, tracking and context

ARCHITECTURE

FastAPI, AgentOS and persistence

AI

Agno, providers and fallback

QUALITY

Tests, i18n and maintainability

That's all, folks!

A Looney Tunes-style sign-off for Series Atlas.

The episode ends here. The next story is already waiting.

SERIES ATLAS

Next episode?

Search. Track. Understand.